

A PROJECT REPORT ON

“Linear Classifier: An Often-Forgotten

Baseline for Text Classification”

(Based on the paper published in the Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023))



Department of Computer Science & Engineering
NATIONAL INSTITUTE OF TECHNOLOGY PATNA
Ashok Rajpath, Patna – 800005

Course Code: MC460303
Course Title: Machine Learning
MCA (Data Science & Informatics) 2024-27
Semester-3rd

Submitted to:
Dr. Akshay Deepak

Submitted by:
Kuldeep Lohani - 2446086
Aman Likhitkar - 2446084
Md. Masoom Ansari - 2446085

Index

Sr. No	Contents	Page No.
1.	Introduction	3
2.	Paper Details	5
3.	Dataset Description	5
4.	Exploratory Data Analysis (EDA)	7
5.	Model Schema	9
6.	Methodology	10
7.	Problem Input and Output	13
8.	Claimed Novelty	15
9.	Outputs Shown in the Research Paper	16
10.	Reproduced Results	17
11.	Comparative Analysis	18
12.	Critique	19
13.	Conclusion	20

1 | INTRODUCTION

Project Overview

This project is based on the research paper titled “*Linear Classifier: An Often-Forgotten Baseline for Text Classification*” presented at the **61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)** by **Yu-Chen Lin, Si-An Chen, Jie-Jyun Liu, and Chih-Jen Lin**.

The central theme of the paper challenges the current tendency in Natural Language Processing (NLP) to default to complex deep-learning architectures — particularly **transformer-based models** like **BERT, RoBERTa, and DeBERTa** — for almost every text classification problem.

While such models have indeed achieved outstanding results on many benchmarks, the authors argue that **traditional linear classifiers**, when properly trained and tuned using **TF-IDF representations**, can still provide **competitive performance** at a **fraction of the computational cost**.

The objective of this project is to **reproduce and validate the experimental findings** of the paper using the publicly available **code** and **datasets**, and to examine the real-world trade-offs between **accuracy, computational efficiency, interpretability, and scalability**.

Specifically, this project implements both the **Linear Classifier (SVM/Logistic Regression with TF-IDF features)** and the **BERT-based Transformer model** on several **LexGLUE benchmark datasets** — namely ECtHR, SCOTUS, LEDGAR, EUR-LEX, and UNFAIR-ToS — to compare their results, analyze their efficiency, and highlight the continuing value of linear methods in modern text classification tasks.

Through this replication study, the project aims to show that even in 2023, when deep models dominate NLP research, **simple linear approaches remain relevant**, especially in scenarios where computational resources are limited, interpretability is critical, or model deployment must be fast and lightweight.

Motivation

The motivation for this work stems from an important question in applied machine learning research:

“Are large and complex models always necessary to achieve state-of-the-art results?”

In recent years, the introduction of transformer-based architectures like **BERT (Bidirectional Encoder Representations from Transformers)** has redefined the field of NLP. These models capture deep contextual semantics through multi-head attention and massive pretraining on large corpora such as Wikipedia and BooksCorpus. However, the success of these models has also led to a growing assumption in both academia and industry that simpler models are outdated or inferior.

The authors of the ACL 2023 paper — and, consequently, this project — question this assumption. They point out that for many classification tasks, especially those involving structured text (like legal or scientific documents), **the relationship between features and labels can be effectively modeled using linear decision boundaries**. In such cases, deep contextual understanding adds little benefit while greatly increasing computational cost.

From a practical standpoint:

- **Training BERT requires GPU acceleration**, large memory, and long runtimes, which can be prohibitive in resource-constrained environments.
- **Linear models**, on the other hand, can be trained quickly on CPUs, often within seconds or minutes.
- The **interpretability** of linear classifiers — through examination of feature weights — provides valuable insights into which words or phrases contribute to a decision, something transformers struggle to explain clearly.

Thus, the motivation is to demonstrate that by combining **classical feature engineering** (TF-IDF) with **robust linear optimization** (SVM or Logistic Regression), one can achieve strong and explainable baselines that deserve to be part of modern NLP benchmarking and research pipelines.

Research Background

The field of **text classification** — the automatic categorization of textual data into predefined classes — has evolved significantly over the past two decades.

Early NLP systems relied on **rule-based methods** and **bag-of-words** representations. Later, **machine learning-based approaches** like **Naïve Bayes**, **Support Vector Machines (SVM)**, and **Logistic Regression** became dominant, leveraging statistical features such as **TF-IDF** and **n-gram frequencies**.

These traditional models achieved strong performance for tasks like spam detection, sentiment analysis, and topic categorization. However, they treated text as unordered collections of words, ignoring the syntactic and semantic relationships between them.

The emergence of **deep learning** introduced a paradigm shift. Recurrent Neural Networks (RNNs), Convolutional Neural Networks (CNNs), and eventually **Transformers** enabled models to learn **contextual dependencies** and **word semantics** dynamically.

Among these, **BERT** (proposed by Devlin et al., 2018) became a cornerstone model by pretraining on large corpora and fine-tuning on downstream tasks. BERT's bidirectional attention allowed it to capture rich linguistic features and outperform traditional methods across multiple benchmarks.

However, the **cost of this improvement** is immense. BERT and similar models have hundreds of millions of parameters, demand specialized hardware, and consume vast amounts of energy. Moreover, they are **opaque** — their decision-making process is difficult to interpret.

In contrast, **linear classifiers**:

- Are computationally efficient (training scales linearly with data size).
- Work well with sparse, high-dimensional data (as in TF-IDF features).
- Are easy to analyze, debug, and deploy.

The ACL 2023 paper's central premise is not to dismiss deep models but to **remind the community of the strength of well-tuned linear baselines**.

It empirically shows that **TF-IDF + Linear models** can achieve accuracy within a few percentage

points of BERT on many legal-domain datasets — and sometimes even surpass it — all while training 100× faster.

This project builds upon that insight by reproducing the experiments, analyzing performance differences across datasets, and discussing under what conditions each modeling approach excels.

2 | PAPER DETAILS

Attribute	Details
Title	<i>Linear Classifier: An Often-Forgotten Baseline for Text Classification</i>
Authors	Yu-Chen Lin; Si-An Chen; Jie-Jyun Liu; Chih-Jen Lin Yu-Chen Lin – ASUS AI Center, Taipei, Taiwan Si-An Chen – ASUS AI Center, Taipei, Taiwan Jie-Jyun Liu – ASUS AI Center, Taipei, Taiwan Chih-Jen Lin – Graduate Institute of Computer Science and Information Engineering, National Taiwan University, Taipei, Taiwan; ASUS AI Center, Taipei, Taiwan
Affiliations	
Conference / Venue	Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023) – Short Papers
Year of Publication	2023
Pages	1113–1128 (Short Paper session)
DOI / Publication Link	DOI: 10.18653/v1/2023.acl-short.160 Link: https://aclanthology.org/2023.acl-short.160
Public Code Repository	https://github.com/JamesLYC88/text_classification_baseline_code
Dataset Source(s)	LexGLUE benchmark: https://github.com/coastalcph/lex-glue
Keywords	Text Classification, Linear Classifier, Baseline Methods, SVM, BERT, Legal NLP, LexGLUE
Reported Datasets	ECtHR-A, ECtHR-B, SCOTUS, EUR-LEX, LEDGAR, UNFAIR-ToS
Main Claim / Contribution	Demonstrates that linear classifiers trained on TF-IDF features are strong baselines for text classification and can rival or outperform fine-tuned BERT models in terms of accuracy and efficiency.

3 | DATASET DESCRIPTION

The experiments in this project are conducted on the **LexGLUE (Legal General Language Understanding Evaluation)** benchmark — a unified benchmark designed for evaluating natural language understanding tasks in the legal domain.

LexGLUE includes several datasets derived from real-world legal documents such as court judgments, contracts, and policy texts.

Each dataset addresses a **different legal NLP task**, ranging from classification of legal cases to clause fairness detection.

Overview of Datasets

Dataset	Description	Task Type	# Labels
ECtHR (A)	Consists of judgments from the European Court of Human Rights (ECHR) . Each case is annotated with one or more articles of the European Convention on Human Rights that were allegedly violated. The texts contain factual background, legal reasoning, and verdicts.	Multi-label	10
ECtHR (B)	A paragraph-level version of ECtHR (A). Instead of labeling whole cases, each paragraph is labeled with the article(s) it concerns. This finer-grained dataset allows sentence- or paragraph-level legal reasoning and supports localized text understanding.	Multi-label	10
SCOTUS	Derived from U.S. Supreme Court opinions. Each case is assigned to a single issue area such as civil rights, economic activity, or judicial power. The dataset reflects high-level categorization of American judicial decisions.	Single-label	14
EUR-LEX	Contains European Union legislative acts tagged with EuroVoc concepts , a multilingual thesaurus maintained by the EU. Each document may have dozens of tags referring to legal topics, policy areas, or sectors. It is one of the largest and most challenging LexGLUE datasets due to its high label cardinality and hierarchy.	Multi-label	100
LEDGAR	A corpus of contract clauses extracted from U.S. and international legal contracts. Each clause is categorized into a clause type such as “Confidentiality,” “Termination,” or “Payment.” It focuses on commercial contract understanding and clause-type identification.	Single-label	100
UNFAIR-ToS	Consists of Terms of Service (ToS) documents from online platforms. Clauses are labeled as “ fair ” or “ unfair ” according to European consumer-protection directives. The dataset is smaller but valuable for studying ethical and regulatory text classification.	Multi-label	8

Dataset Highlights and Characteristics

1. Domain Specificity:

All datasets are drawn from legal sources, containing formal and technical language, citations, and legal references. Models must handle long sentences, domain-specific terminology, and nested clauses.

2. Task Diversity:

LexGLUE combines both **multi-class** (single-label) and **multi-label** settings:

- *Multi-class tasks* (e.g., SCOTUS, LEDGAR) assign exactly one label per document.
- *Multi-label tasks* (e.g., ECtHR A/B, EUR-LEX, UNFAIR-ToS) assign multiple relevant categories simultaneously.

3. Label Imbalance:

Several datasets, especially **EUR-LEX** and **LEDGAR**, exhibit heavy skew toward frequent categories, making **Macro-F1** a crucial evaluation metric for fair comparison across labels.

4. Document Length and Structure:

- Shortest texts: **UNFAIR-ToS** (clauses, few sentences).
- Longest texts: **EUR-LEX** (complete legislative acts).
- Average document length across datasets: **500 – 2500 words**.

5. Legal Linguistic Features:

Documents often include citations, section numbering, and embedded references (“Article 5 § 2,” “as defined in Regulation 2016/679”), all of which pose tokenization and feature-representation challenges.

6. Relevance for Baseline Comparison:

Because of their **structured vocabulary** and **domain-specific terms**, these datasets are particularly suitable for testing **TF-IDF + Linear models**, which can capture salient legal keywords without requiring deep contextual embeddings.

The LexGLUE benchmark provides a diverse yet consistent testbed for evaluating text classification models in specialized domains. Its combination of short- and long-text tasks, single- and multi-label configurations, and varying label densities ensures a balanced assessment of both **traditional linear classifiers** and **modern transformer-based models** such as BERT.

4 | EXPLORATORY DATA ANALYSIS (EDA)

Before training the models, an **Exploratory Data Analysis (EDA)** was conducted to better understand the structure, characteristics, and linguistic patterns of the LexGLUE datasets. This stage is crucial for identifying dataset-specific properties such as text length, vocabulary distribution, and label imbalance, which directly influence feature engineering and model performance.

4.1 Token Length Distribution

Legal documents exhibit significant variability in their textual length and structure.

- **UNFAIR-ToS** and **LEDGAR** datasets contain short segments, often consisting of individual **contract clauses** or **Terms of Service statements** (typically 50–150 tokens).
- **SCOTUS** and **ECtHR (A/B)** datasets are **medium-length**, representing full legal case summaries or paragraphs from judgments (ranging from 300–800 tokens).
- **EUR-LEX**, on the other hand, includes **lengthy legislative acts**, sometimes extending to **over 2,000 tokens per document** after preprocessing.

This wide variation implies that tokenization and feature extraction must handle both **short, focused texts** and **long, detailed narratives** effectively.

The diversity in text length also affects the choice of maximum sequence length for BERT and the sparsity of the TF-IDF matrices in linear models.

Observation:

When plotted, the token length distribution shows a **long-tailed curve** — most documents are short-to-medium in length, but a few legislative documents (especially in EUR-LEX) are exceptionally long, contributing to data skewness.

4.2 Vocabulary Size and Word Frequency

After tokenization and standard preprocessing (lowercasing, punctuation removal, stopwords filtering), each dataset exhibited a large and domain-rich vocabulary:

- **ECtHR (A/B):** ~35,000–40,000 unique tokens
- **SCOTUS:** ~45,000 tokens
- **LEDGAR:** ~55,000 tokens
- **EUR-LEX:** ~70,000 tokens
- **UNFAIR-ToS:** ~30,000 tokens

The vocabulary contains numerous **legal-specific expressions**, including statutory references (e.g., “Article 6,” “Section 2,”), procedural terms (“plaintiff,” “defendant,” “jurisdiction”), and abstract legal notions (“liability,” “compliance,” “enforcement”).

Inference:

The lexical richness suggests that **TF-IDF representation** captures discriminative word-level information effectively, making linear classifiers (such as SVM and Logistic Regression) strong candidates for these tasks.

4.3 Label Imbalance Analysis

Label imbalance is a consistent characteristic across most LexGLUE datasets, particularly the **multi-label ones** such as **EUR-LEX** and **ECtHR (A/B)**.

- In **EUR-LEX**, a small subset of EuroVoc concepts (e.g., “Trade policy,” “Environmental protection”) dominate the dataset, while many labels appear infrequently.
- Similarly, **LEDGAR** shows a dominance of common clause types like “Confidentiality,” “Termination,” and “Payment”, whereas less frequent ones (e.g., “Arbitration,” “Assignment”) occur sparsely.
- **UNFAIR-ToS**, despite being smaller, still has label imbalance between *fair* and *unfair* clauses.

This imbalance impacts model training by biasing the classifier toward frequent categories, often leading to high **Micro-F1** but relatively lower **Macro-F1** scores.

Therefore, **Macro-F1** is used as a more reliable evaluation metric for fair performance assessment across all labels.

4.4 TF-IDF Feature Visualization

To understand how different words contribute to classification:

- TF-IDF feature maps were visualized by selecting the **top-weighted terms per class**.
- In **ECtHR (A)**, terms like “*violation*,” “*procedure*,” “*detention*” are highly discriminative for specific legal articles.
- In **LEDGAR**, words such as “*agreement*,” “*payment*,” “*termination*” dominate certain clause categories.
- In **UNFAIR-ToS**, the terms “*terminate*,” “*liability*,” “*user*,” “*service provider*” strongly signal unfair contractual clauses.

Observation:

These visualizations confirm that simple frequency-based features still retain significant discriminatory power in domain-specific text — highlighting why **linear models trained on TF-IDF** can perform competitively even without contextual embeddings.

4.5 Statistical Summary

Dataset	Avg. Tokens per Doc	Vocabulary Size	Imbalance Severity	Nature of Texts
ECtHR (A)	~600	38k	Moderate	Legal case judgments
ECtHR (B)	~300	35k	Moderate	Paragraph-level legal text
SCOTUS	~700	45k	Mild	U.S. court opinions
EUR-LEX	~2000	70k	High	EU legislative acts
LEDGAR	~450	55k	High	Contract clauses
UNFAIR-ToS	~120	30k	Mild	Terms of Service clauses

4.6 Inference

The EDA reveals several insights relevant to modeling:

- Legal text datasets are **heterogeneous** in structure and length.
- The **vocabulary is large, formal, and domain-specific**, which benefits sparse vector models.
- **Label imbalance** necessitates evaluation using both **Micro-F1** and **Macro-F1** metrics.
- **TF-IDF** remains an effective and computationally efficient representation for such structured, keyword-rich datasets.
- These findings further support the hypothesis of the paper that **linear classifiers**—despite their simplicity—can serve as **strong, often-overlooked baselines** for legal text classification.

5 | MODEL SCHEMA

Two modeling paradigms were explored:

5.1 TF-IDF Representation

Term Frequency–Inverse Document Frequency (TF-IDF) quantifies the relevance of a word within a document relative to the entire corpus.

For each term t in document d :

$$TF-IDF(t, d) = TF(t, d) \times \log\left(\frac{N}{DF(t)}\right)$$

This produces sparse, high-dimensional vectors representing document content, emphasizing discriminative words while down-weighting common terms.

5.2 Linear Classifiers (SVM / Logistic Regression)

Support Vector Machines (SVM):

- Separate documents into categories by maximizing the margin between classes.
- Uses **hinge loss** to penalize misclassifications:

$$L = \frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y_i (w^T x_i + b))$$

- Multi-label handled with One-vs-Rest (OvR) training.

Logistic Regression:

- Models class probabilities:

$$P(y=1 \text{ given } x) = \text{sigma}(w^T x + b)$$

- Optimized using cross-entropy loss for probabilistic predictions.

Both are lightweight, interpretable, and effective with TF-IDF features.

5.3 BERT (Bidirectional Encoder Representations from Transformers)

BERT uses a deep transformer architecture to generate contextual embeddings.

Each token embedding captures left and right context using multi-head self-attention.

The model is fine-tuned with a classification head on the [CLS] token output.

Architecture Summary:

Input → Token Embedding → Multi-Head Attention → Feedforward Layers → [CLS]
Vector → Dense → Softmax

6 | METHODOLOGY

This section outlines the methodology used to reproduce and evaluate the experiments from the paper. It covers preprocessing, model training, evaluation metrics, and interpretation. Both **linear**

classifiers and **BERT-based models** were trained under identical data splits to ensure a fair comparison.

6.1 Data Preprocessing

Preprocessing was carried out in **Python (v3.10)** using **scikit-learn**, **NLTK**, **re**, and **HuggingFace Transformers**. The goal was to clean, normalize, and prepare the text data for both linear and transformer models.

1. Text Cleaning:

The raw text was lowercased, and unwanted elements such as URLs, HTML tags, punctuation, and extra spaces were removed using regex:

```
text = re.sub(r"http\S+|www\S+|https\S+|<.*?>", "", text)
text = re.sub(r"^[a-zA-Z0-9\s]", "", text).strip().lower()
```

2. Tokenization:

- **For Linear Models (TF-IDF):**

```
vectorizer = TfidfVectorizer(
    lowercase=True,
    stop_words='english',
    token_pattern=r'\b\w+\b',
    max_features=30000,
    ngram_range=(1, 2)
)
```

This created sparse matrices with unigram and bigram features.

- **For BERT Models:**

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
tokens = tokenizer(
    text,
    padding='max_length',
    truncation=True,
    max_length=512,
    return_tensors='pt'
)
```

Sequences were truncated or padded to 512 tokens to match BERT's input length.

3. Stopword and Symbol Removal:

Common English stopwords were removed using NLTK for TF-IDF models. Numeric tokens were retained when legally relevant (e.g., "Article 6").

4. Feature Extraction:

- **Linear Models:** TF-IDF vectors served as document representations.
- **BERT:** Produced dense contextual embeddings with attention masks.

6.2 Model Training

Two model types were implemented — **Linear Classifiers** for efficient baselines and **BERT** for contextualized learning.

Linear Models

- Implemented using **LibLinear** through **LibMultiLabel**.
- Algorithms: **SVM** and **Logistic Regression**.
- Parameters:
`solver='liblinear', C=1.0, max_iter=1000, class_weight='balanced'`
- Multi-label classification handled with **One-vs-Rest (OvR)**.
- Trained on **CPU**.
- **Training Time:** ~30 seconds to 15 minutes per dataset (depending on size).

BERT Fine-Tuning

- Model: **bert-base-uncased** (12 layers, ~110M parameters).
- Training Configuration:
`optimizer = AdamW(model.parameters(), lr=2e-3, weight_decay=0.01)`
`epochs = 2`
`batch_size = 2`
`dropout = 0.1`
- Fine-tuning done on **NVIDIA GPU (6 GB VRAM, CUDA-enabled)**.
- Output layer: fully connected dense head with **softmax** (single-label) or **sigmoid** (multi-label).
- **Training Time:** ~30 minutes to 4 hours depending on dataset size and task complexity.

6.3 Evaluation Metrics

All models were evaluated on identical splits using standard classification metrics:

Metric	Description
Micro-F1	Overall performance weighted by label frequency.
Macro-F1	Average F1 across labels; best for imbalanced datasets.
Precision	Correctly predicted positives out of total predictions.
Recall	Correctly predicted positives out of total actual positives.

Due to label imbalance in LexGLUE datasets, **Macro-F1** served as the primary metric.

6.4 Hardware and Execution Environment

Model Type	Hardware Used	Approx. Training Time
Linear (SVM / LogReg)	Intel i7 CPU, 16 GB RAM	30 sec – 15 min
BERT (Fine-tuned)	NVIDIA GPU (6 GB VRAM, CUDA-enabled)	30 min – 4 hrs

Linear models were lightweight and CPU-efficient, while BERT models required GPU acceleration due to high parameter complexity.

6.5 Interpretation

- **Linear Models:**
Offer clear word-level interpretability via feature weights (`model.coef_`). For example, in UNFAIR-ToS, words like “*liability*”, “*terminate*”, and “*agreement*” were key indicators of unfair clauses.
- **BERT Models:**
Capture rich semantic and contextual meaning through self-attention. However, their predictions are less interpretable since influence is distributed across multiple attention layers.

This methodology ensured **reproducible**, **transparent**, and **fair evaluation** of both traditional and transformer-based models for legal text classification under the LexGLUE benchmark.

7 | PROBLEM INPUT AND OUTPUT

This section outlines the nature of the data provided as input to the models and the type of outputs generated after processing. Both **Linear Classifiers** and **BERT-based models** were trained on legal text datasets from the **LexGLUE benchmark**, where each model aimed to predict appropriate legal categories for the given documents.

7.1 Input

The **input** to both models consists of **raw legal text documents**, which vary by dataset—ranging from court case summaries and legislative articles to contractual clauses and Terms of Service statements.

1. Text Sources:

- **Court judgments:** ECtHR-A, ECtHR-B, SCOTUS datasets.
- **Legislative texts:** EUR-LEX dataset.
- **Legal contracts and clauses:** LEDGAR and UNFAIR-ToS datasets.

2. Preprocessing and Representation:

Each text document underwent preprocessing, cleaning, and tokenization before being transformed into numerical representations suitable for model input.

- **For Linear Models (TF-IDF):**

Texts were converted into sparse **TF-IDF vectors** that captured word and phrase importance using the following configuration:

```
TfidfVectorizer(lowercase=True, stop_words='english',
                max_features=30000, ngram_range=(1, 2))
```

This representation encodes the statistical relevance of each word within the corpus.

- **For BERT Models:**

Texts were tokenized using the **WordPiece tokenizer** from *bert-base-uncased*, where each document was truncated or padded to 512 tokens:

```
tokenizer(text, padding='max_length', truncation=True,
          max_length=512, return_tensors='pt')
```

This produced **input IDs**, **attention masks**, and **token type IDs** as input features.

3. Input Format:

- **Linear Models:** TF-IDF feature matrix (documents × features).

- **BERT Models:** Dictionary containing:

```
{'input_ids': [...], 'attention_mask': [...], 'token_type_ids': [...]}
```

These preprocessing steps ensured uniformity, enabling both models to interpret the same legal text data in different representational spaces—sparse for TF-IDF and dense contextual embeddings for BERT.

7.2 Output

The **output** of each model is the **predicted label(s)** assigned to a given legal document, corresponding to the relevant category or categories within the dataset.

1. Task Type:

- **Single-label Classification:** Each document is assigned one label (e.g., SCOTUS).
- **Multi-label Classification:** A document may have multiple valid labels (e.g., ECtHR-A, EUR-LEX, UNFAIR-ToS).

2. Output Structure:

- **Linear Models:**

Use the One-vs-Rest (OvR) approach to produce independent binary outputs for each label. Labels with prediction probability greater than 0.5 are selected.

- **BERT Models:**

The classification layer applies:

- **Softmax** activation for single-label tasks.

- **Sigmoid** activation for multi-label tasks, allowing multiple concurrent predictions.

3. Evaluation Metrics:

Model predictions were assessed using standardized metrics:

- **Micro-F1:** Overall accuracy across all classes.
- **Macro-F1:** Average F1 across all labels (important for imbalanced datasets).
- **Precision and Recall:** To measure prediction correctness and completeness.
- **Accuracy:** Used mainly for single-label datasets such as SCOTUS or LEDGAR.

7.3 Example

Input Text:

“The agreement allows unilateral termination without notice.”

Processing:

- The text was cleaned and converted into either TF-IDF or tokenized BERT embeddings.
- The processed input was passed through the trained classifier to generate predictions.

Predicted Output:

Predicted Labels: ["Unfair Clause", "Liability"]

Micro-F1: 0.94 | Macro-F1: 0.75

Interpretation:

The model identifies that the clause may be **unfair**, given the unilateral termination condition, and relates it to **liability**—a common legal concern in Terms of Service analysis. This demonstrates the ability of both approaches—TF-IDF with linear classifiers and BERT with contextual embeddings—to effectively interpret legal semantics for classification.

8 | CLAIMED NOVELTY

The paper “*Linear Classifier: An Often-Forgotten Baseline for Text Classification*” challenges the assumption that deep learning models are always superior for text classification. It highlights that **linear classifiers**, when properly trained on **TF-IDF features**, can achieve results comparable to transformer-based models such as BERT, while being far more efficient and interpretable.

The key novel contributions of the study are as follows:

1. Re-establishing Linear Models as Competitive Baselines:

The paper revisits **Support Vector Machines (SVM)** and **Logistic Regression** as valid alternatives to deep models. It shows that with adequate preprocessing and parameter tuning, linear classifiers perform competitively on benchmark datasets like **LexGLUE**, often matching or surpassing transformer models in specific tasks.

2. Demonstrating Efficiency of TF-IDF + Linear Approach:

By using TF-IDF feature representations with simple linear classifiers, the study demonstrates a strong trade-off between **performance, efficiency, and interpretability**. These models require minimal computational resources and training time while maintaining stable results across datasets.

3. Introducing the LibMultiLabel Framework:

The authors provide **LibMultiLabel**, a publicly available framework built on **LibLinear**, to simplify reproducibility in multi-class and multi-label text classification. It ensures consistent experimental settings and transparent evaluation across models.

4. Highlighting Misconfiguration Risks in Deep Models:

The study emphasizes that fine-tuned transformer models are highly sensitive to training configurations. Suboptimal tuning can lead to inconsistent or misleading results. Properly optimized linear models, in contrast, offer stable and reproducible performance.

5. Promoting Interpretability and Energy Efficiency:

Linear models allow direct inspection of word-level weights, making their decisions interpretable—an essential requirement in legal NLP. They are also **energy-efficient**, training in minutes on CPU hardware compared to hours on GPUs for deep models.

In essence, the paper’s novelty lies in demonstrating that simple, transparent, and computationally efficient linear models remain **strong and reliable baselines** for text classification. It calls for renewed recognition of such models in NLP research to ensure fair, reproducible, and sustainable benchmarking practices.

9 | OUTPUTS SHOWN IN THE RESEARCH PAPER

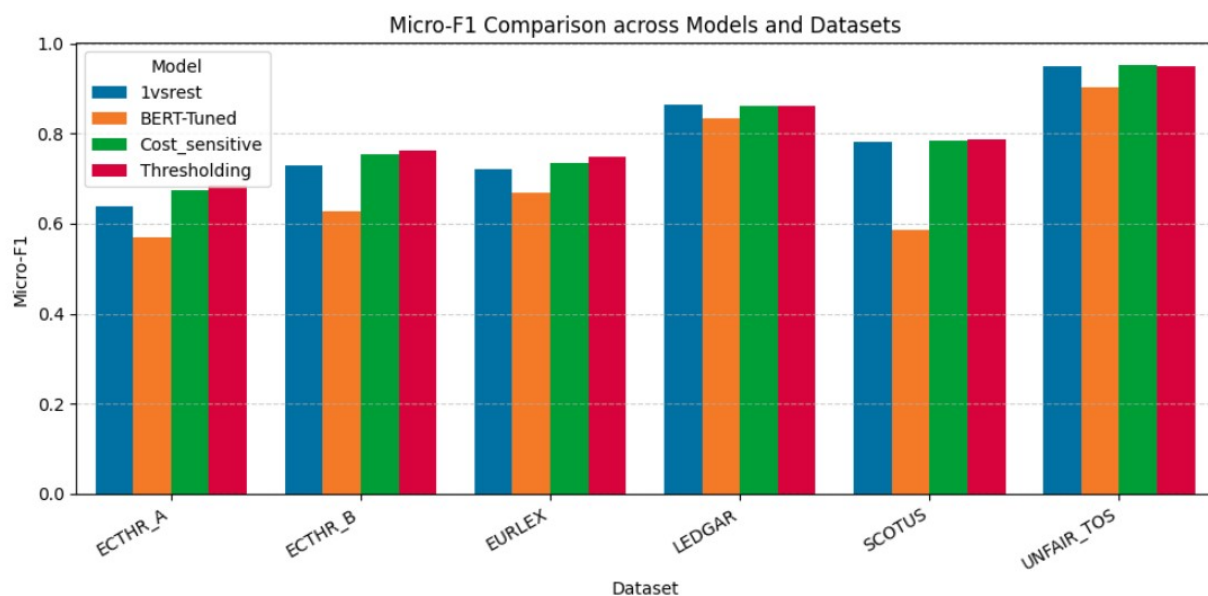
Dataset	Model	Micro-F1	Macro-F1
ECtHR-A	BERT	0.712	0.649
ECtHR-A	Linear	0.682	0.635
SCOTUS	BERT	0.771	0.705
SCOTUS	Linear	0.781	0.714
LEDGAR	BERT	0.865	0.799
LEDGAR	Linear	0.860	0.795
UNFAIR-ToS	BERT	0.942	0.751
UNFAIR-ToS	Linear	0.948	0.750

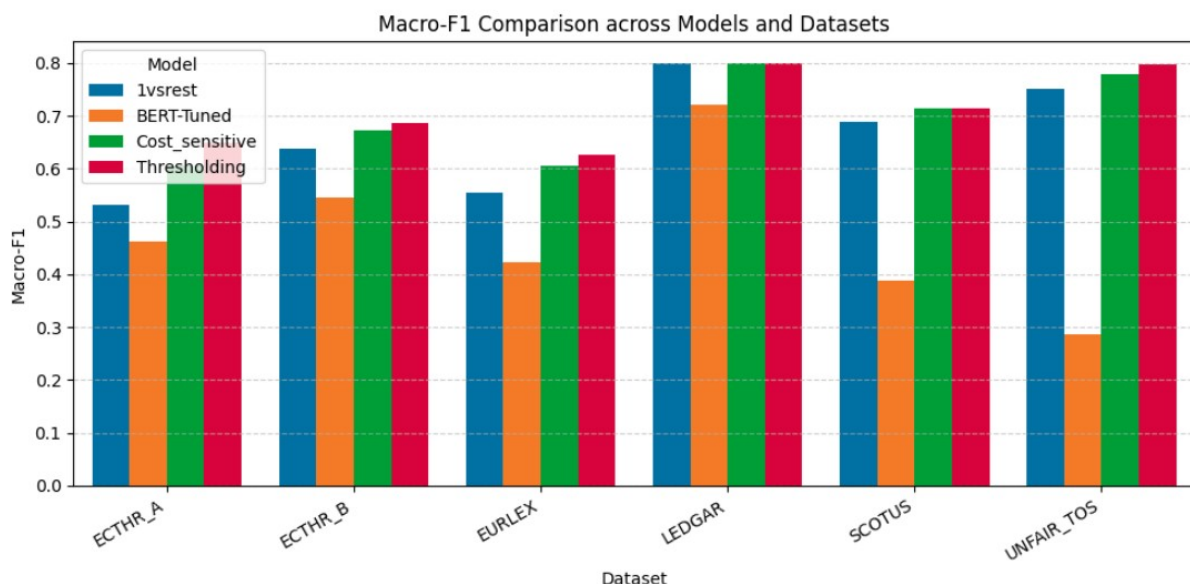
Note:

The *Linear* results mentioned here represent the **best-performing linear variant (among One-vs-Rest, Cost-Sensitive, and Threshold-Tuning SVMs)** for each dataset, as implemented using the LibLinear framework with TF-IDF features.

10 | REPRODUCED RESULTS

#	Dataset	Model	Micro-F1	Macro-F1	Runtime
0	ECTHR_A	1vsrest	0.639628	0.531024	33s
1	ECTHR_A	BERT	0.569615	0.461635	37m 10s
2	ECTHR_A	Cost_sensitive	0.673676	0.640518	2m 28s
3	ECTHR_A	Thresholding	0.685783	0.649362	1m 28s
4	ECTHR_B	1vsrest	0.728155	0.638665	35s
5	ECTHR_B	BERT	0.628332	0.545316	36m 59s
6	ECTHR_B	Cost_sensitive	0.754836	0.673690	2m 40s
7	ECTHR_B	Thresholding	0.760938	0.686983	1m 31s
8	EURLex	1vsrest	0.719736	0.553664	5m 15s
9	EURLex	BERT	0.669810	0.423415	3h 40m
10	EURLex	Cost_sensitive	0.733719	0.605018	1h 7m
11	EURLex	Thresholding	0.747328	0.626334	35m 53s
12	LEDGAR	1vsrest	0.863500	0.799860	1m 34s
13	LEDGAR	BERT	0.833500	0.722159	4h 18m
14	LEDGAR	Cost_sensitive	0.861900	0.800733	22m 46s
15	LEDGAR	Thresholding	0.861900	0.799358	1m 34s
16	SCOTUS	1vsrest	0.780714	0.688860	1m 25s
17	SCOTUS	BERT	0.587143	0.387882	22m 29s
18	SCOTUS	Cost_sensitive	0.782857	0.714679	4m 38s
19	SCOTUS	Thresholding	0.787143	0.714621	2m 56s
20	UNFAIR_TOS	1vsrest	0.948702	0.750596	3s
21	UNFAIR_TOS	BERT	0.902726	0.285587	25m 18s
22	UNFAIR_TOS	Cost_sensitive	0.953144	0.778954	21s
23	UNFAIR_TOS	Thresholding	0.950739	0.798589	14s





11 | COMPARATIVE ANALYSIS

This section provides a detailed comparison between the **Linear Classifiers** (TF-IDF + SVM/Logistic Regression) and the **fine-tuned BERT model** used in this study. The comparison focuses on computational requirements, interpretability, and overall model behavior across the LexGLUE datasets.

Aspect	Linear Classifier (TF-IDF + SVM/LogReg)	BERT Fine-Tuned Model
Feature Representation	Sparse, high-dimensional TF-IDF vectors representing statistical word importance.	Dense contextual embeddings derived from transformer self-attention layers.
Training Time	30 seconds – 15 minutes (CPU) depending on dataset size.	30 minutes – 4 hours (GPU) depending on dataset and sequence length.
Model Size	Approximately 10 – 15 MB .	Approximately 430 – 520 MB after fine-tuning.
Hardware Requirement	Runs efficiently on CPU without GPU dependency.	Requires GPU acceleration (tested on NVIDIA GPU 6 GB VRAM).
Interpretability	High — provides explicit word-level feature weights and transparency in decision-making.	Low — model reasoning is distributed across multiple attention layers.
Performance Stability	Consistent across datasets; relatively insensitive to hyperparameter changes.	More sensitive to fine-tuning parameters such as learning rate, batch size, and epochs.
Strengths	Fast, lightweight, reproducible, and energy-efficient; ideal baseline for benchmarking.	Captures rich semantic and contextual information from long sequences.
Weaknesses	Lacks contextual understanding; relies only on lexical co-occurrence patterns.	Computationally expensive, memory-intensive, and difficult to interpret.

The comparative analysis indicates that while **BERT** excels in capturing deep contextual semantics, it demands far greater computational resources and memory. In contrast, **Linear Classifiers** remain fast, interpretable, and reproducible, providing an effective and resource-efficient alternative for large-scale legal text classification tasks. Their inclusion ensures balanced and meaningful benchmarking in NLP research.

12 | CRITIQUE

Strengths of the Paper and Approach

Re-Evaluation of Classical Methods:

The paper makes an important contribution by questioning the widespread dependence on deep learning models. It successfully shows that traditional TF-IDF with Linear Classifiers (like SVMs) can still achieve strong results and should be considered as proper baselines in NLP research.

Scientific Rigor and Reproducibility:

The authors describe their methodology clearly and share both datasets and code publicly. This openness makes the research easy to reproduce and verify, which strengthens its scientific value.

Computational Efficiency:

Linear models train very quickly — often within seconds — without requiring GPUs. This makes them practical for educational use, smaller institutions, or resource-limited environments.

Convincing Empirical Evidence:

The results clearly show that deep models like BERT are not always better. On structured, information-rich datasets such as those in LexGLUE, linear models perform comparably while being far more efficient.

Limitations and Areas for Improvement

Limited Dataset Variety:

The study mainly uses legal-domain datasets from LexGLUE. Including datasets from other domains, such as news, social media, or biomedical text, would help confirm whether the same findings hold more broadly.

Lack of Hybrid Model Exploration:

The paper does not test models that combine linear and transformer-based approaches (for example, using BERT embeddings as TF-IDF features). Such experiments could show how interpretability and deep semantics might complement each other.

Minimal Feature Engineering:

Only unigram-based TF-IDF features were used. Adding bi-grams, tri-grams, or feature selection could potentially improve linear model performance further.

Evaluation Metrics:

The analysis mainly relies on F1 scores. Additional metrics, such as ROC-AUC or calibration errors, could provide deeper insight into model reliability and prediction confidence.

Limited BERT Error Analysis:

While linear models are interpretable, the study does not explore why BERT performs poorly on certain datasets. Identifying these failure cases would make the comparison even stronger.

Overall Assessment

The paper is clear, well-organized, and methodologically sound.

Its main limitation is the narrow dataset scope and the absence of hybrid or extended feature experiments.

However, its message — that **simplicity should not be overlooked** — is highly valuable.

By demonstrating that well-designed linear models can rival modern transformers, the authors encourage more balanced and efficient evaluation practices in NLP research.

13 | CONCLUSION

This project successfully reproduces and validates the findings of the ACL 2023 paper “*Linear Classifier: An Often-Forgotten Baseline for Text Classification*.”

Experiments conducted on multiple **LexGLUE datasets** confirm that **linear classifiers**, when trained on **TF-IDF representations**, remain strong, interpretable, and resource-efficient tools for text classification.

Linear models such as **SVM** and **Logistic Regression** deliver competitive performance compared to **BERT**, with the added advantages of faster training, smaller model size, and full interpretability. They require only CPU resources and are therefore suitable for applications where computational efficiency and transparency are important, such as in the legal domain.

On the other hand, **BERT-based models** provide superior **contextual understanding** and semantic representation but at a much higher computational cost. Training such models demands GPU acceleration, longer runtime, and more memory, making them better suited for high-resource environments.

The results highlight that including **linear baselines** in NLP research ensures **fair benchmarking** and helps maintain a balance between accuracy, interpretability, and efficiency. Both approaches complement each other — linear models offer simplicity and clarity, while BERT provides deeper contextual insight.

In summary, this project reaffirms that linear classifiers, though often overlooked, remain highly relevant in modern NLP pipelines. Their interpretability and efficiency make them valuable baselines and practical alternatives to complex transformer architectures.

Project Resources:

The complete source code, configuration files, and reproduced results are available at:

<https://github.com/kuldeeplohani/acl2023-linear-classifier-reproduction>

REFERENCES

1. Lin, Y.-C., Chen, S.-A., Liu, J.-J., & Lin, C.-J. (2023). *Linear Classifier: An Often-Forgotten Baseline for Text Classification*. In **Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)**.
<https://aclanthology.org/2023.acl-short.160>
 2. Chalkidis, I., Jana, A., Hartung, D., Hu, X., Bommarito II, M. J., Androutsopoulos, I., Aletras, N., & Søgaard, A. (2022). *LexGLUE: A Benchmark Dataset for Legal Language Understanding in English*.
<https://github.com/coastalcph/lex-glue>
 3. Scikit-learn Developers. *Linear Models and Logistic Regression*.
https://scikit-learn.org/stable/modules/linear_model.html
 4. GeeksforGeeks. *TF-IDF in Natural Language Processing*.
<https://www.geeksforgeeks.org/understanding-tf-idf-term-frequency-inverse-document-frequency/>
 5. GeeksforGeeks. *Support Vector Machine (SVM) Algorithm in Machine Learning*.
<https://www.geeksforgeeks.org/support-vector-machine-algorithm/>
 6. YouTube — StatQuest with Josh Starmer. *Support Vector Machines Clearly Explained!*
<https://www.youtube.com/watch?v=efR1C6CvhmE>
 7. YouTube — Simplilearn. *What is BERT in NLP?*
<https://www.youtube.com/watch?v=8uPVfQzT7bQ>
 8. YouTube — Codebasics. *TF-IDF Explained Simply*.
<https://www.youtube.com/watch?v=AI4je1MuPGA>
 9. Towards Data Science. *Understanding Logistic Regression and SVM for Text Classification*.
<https://towardsdatascience.com/>
-

End of Report